

PROGETTO DI FONDAMENTI DI INTELLIGENZA ARTIFICIALE

HEART

Hospital Environment Adaptive Routing Technique

(Tecnica di instradamento adattivo in ambiente ospedaliero)

Irene Iuorio

January 2026

Contents

0.1	Codice sorgente	3
1	Introduzione	3
2	Domanda iniziale	3
3	Ambiente e problema di pianificazione	3
3.1	Specifica PEAS	4
4	Algoritmi di ricerca utilizzati	4
4.1	Ricerca a Costo Uniforme (Uniform Cost Search)	4
4.2	Algoritmo A*	5
4.3	Relazione tra A* e Ricerca a Costo Uniforme	5
4.4	Euristica euclidea	6
4.5	Ruolo degli algoritmi nel progetto	6
5	Simulazione dei costi di percorrenza	7
5.1	Modello del tempo di percorrenza	7
5.2	Tipo di corridoio	7
5.3	Affollamento e congestione	7
5.4	Implementazione delle componenti stocastiche	8
5.5	Costi statici	8
6	Generazione del dataset	9
6.1	Struttura dei campioni	9
6.2	Campionamento stratificato	9
6.3	Ruolo della tipologia di corridoio	10
6.4	Mescolamento e indipendenza dei campioni	10
6.5	Suddivisione in training e test	10
6.6	Obiettivo del dataset	10
7	Modelli di machine learning per la stima dei costi	10
7.1	Input e output dei modelli	11
7.2	Regressione lineare	11
7.3	Random Forest	11
7.4	Valutazione delle prestazioni	11
7.5	Integrazione con l'algoritmo di pianificazione	12
8	Metriche di valutazione	12
8.1	Metriche a livello di singolo percorso	12
8.2	Metriche aggregate	13
8.3	Confronto tra configurazioni	13
8.4	Quando il machine learning è vantaggioso	13
8.5	Scenari sperimentali e ambiente di test	13
8.6	Scenario normale	14
8.7	Scenario estremo	14
8.8	Discussione complessiva e risposta alla domanda iniziale	16
8.9	Visualizzazione e demo interattiva	16
8.9.1	Esempio Demo: Ore 14:00, Affollamento:90%	17

0.1 Codice sorgente

Il codice del progetto, comprensivo della generazione dei dati, delle implementazioni degli algoritmi di pianificazione, dei modelli di machine learning e della demo interattiva, è disponibile su GitHub.

<https://github.com/ireneiuorio/H.E.A.R.T-FIA.git>

1 Introduzione

Gli algoritmi di ricerca informata, come A^* , sono molto utilizzati per affrontare problemi di pianificazione e navigazione, in cui l'obiettivo è individuare un percorso ottimale tra uno stato iniziale e uno stato finale. In scenari ideali si assume che i costi associati alle azioni siano noti, stabili nel tempo e affidabili; in queste condizioni, A^* è in grado di garantire sia la completezza sia l'ottimalità della soluzione trovata.

Nella realtà, però, queste premesse risultano spesso poco realistiche. In molti contesti concreti, infatti, i costi delle azioni dipendono da fattori dinamici e difficili da prevedere, come l'orario, il livello di affollamento o la presenza di congestione. Di conseguenza, basarsi su costi fissi o medi può indurre l'algoritmo a prendere decisioni che sono ottimali solo dal punto di vista teorico, ma poco efficaci quando applicate al mondo reale.

In situazioni di questo tipo, il machine learning viene spesso utilizzato per stimare i costi delle azioni a partire dai dati osservati. L'obiettivo è fornire all'algoritmo di pianificazione informazioni più aderenti alla realtà, sostituendo o affiancando i costi statici con stime apprese automaticamente.

2 Domanda iniziale

Quando si utilizza un algoritmo di ricerca come A^* , è naturale pensare che fornire informazioni più accurate sui costi delle azioni porti automaticamente a soluzioni migliori. Nella pratica, tuttavia, questo risultato non è sempre garantito, soprattutto quando tali informazioni non sono osservate direttamente ma ricavate da stime.

La domanda centrale da cui parte questo progetto è quindi la seguente:

L'utilizzo di costi stimati tramite machine learning migliora la pianificazione di A^ , oppure in alcuni casi può peggiorarne il comportamento rispetto all'uso di costi statici?*

Per affrontare questa domanda, l'algoritmo di ricerca viene mantenuto invariato e utilizzato come punto di riferimento. A variare è invece il modo in cui vengono forniti i costi degli archi: in alcune configurazioni si adottano costi medi statici, mentre in altre si utilizzano costi stimati tramite modelli di machine learning con diversi livelli di complessità.

Questo approccio consente di focalizzarsi non solo sull'accuratezza delle singole stime, ma soprattutto sull'impatto che tali stime hanno sulle decisioni globali dell'algoritmo di ricerca.

3 Ambiente e problema di pianificazione

Il problema affrontato in questo progetto riguarda la pianificazione di un percorso all'interno di un ambiente indoor, modellato come un grafo non orientato $G = (V, E)$. I nodi del grafo rappresentano punti significativi della struttura, mentre gli archi corrispondono ai corridoi che collegano tali punti.

Lo scenario considerato simula il trasporto di emergenza di un paziente dall'ingresso dell'ospedale verso un reparto specifico. In questo contesto, la rapidità del percorso riveste un ruolo cruciale e il tempo di attraversamento dei corridoi costituisce il principale costo di pianificazione.

A ciascun arco sono associati una lunghezza fisica e un tempo di percorrenza, che rappresentano il costo dell'azione di attraversamento. Questo costo, tuttavia, non è costante: il tempo necessario per percorrere un corridoio dipende da diversi fattori dinamici, come l'orario della giornata, il livello di affollamento e la tipologia del corridoio stesso.

I corridoi sono classificati in tre categorie:

- *centrali*

- *secondari*
- *isolati*

L'ambiente ospedaliero è stato scelto come caso di studio perché rappresenta un contesto realistico in cui i costi di attraversamento variano nel tempo e non sono completamente osservabili. In situazioni di emergenza, l'utilizzo di costi statici o medi può portare a decisioni di pianificazione non ottimali, rendendo necessario il ricorso a stime più precise.

L'obiettivo dell'agente è pianificare un percorso dal nodo di partenza, corrispondente all'ingresso dell'ospedale, al nodo di destinazione, che rappresenta il reparto di interesse, minimizzando il tempo totale di percorrenza in presenza di informazione incerta sui costi.

3.1 Specifica PEAS

Il problema di pianificazione considerato può essere descritto tramite la specifica PEAS, adattata allo scenario di trasporto di emergenza simulato nel progetto.

Performance Le prestazioni dell'agente vengono valutate principalmente in base al tempo totale necessario per trasportare il paziente dall'ingresso dell'ospedale al reparto di destinazione. Si osservano inoltre il numero di nodi espansi e il tempo di esecuzione dell'algoritmo di ricerca, con lo scopo di analizzare l'impatto delle diverse **stime di costo** sul comportamento di A*.

Environment L'ambiente è un ospedale modellato come un grafo non orientato, in cui i corridoi presentano costi di attraversamento variabili nel tempo. Il tempo reale di percorrenza dipende da fattori dinamici quali l'orario, il livello di affollamento e la tipologia del corridoio. L'ambiente risulta quindi dinamico e solo parzialmente osservabile, poiché i costi reali non sono noti direttamente durante la fase di pianificazione.

Actuators L'agente non controlla direttamente il movimento fisico, ma pianifica una sequenza di corridoi da attraversare. Ogni azione consiste nella scelta del prossimo arco del grafo da percorrere.

Sensors L'agente dispone delle informazioni strutturali sul grafo e di un insieme di variabili osservabili, come la lunghezza del corridoio, l'orario, una stima dell'affollamento e il tipo di corridoio. Queste informazioni vengono utilizzate per stimare i costi di attraversamento tramite modelli di machine learning o mediante valori medi predefiniti.

4 Algoritmi di ricerca utilizzati

Per la pianificazione del percorso all'interno dell'ospedale vengono adottate due strategie di ricerca basate sull'algoritmo A*: la **ricerca a costo uniforme (Uniform Cost Search)** e **A* con euristica euclidea**.

La ricerca a costo uniforme rappresenta il caso in cui l'algoritmo prende decisioni esclusivamente sulla base del costo cumulativo dei percorsi già esplorati. In questa configurazione, A* viene impiegato con un'euristica nulla e il suo comportamento coincide con quello dell'algoritmo di Dijkstra.

A* con euristica euclidea introduce invece un'informazione aggiuntiva che guida l'esplorazione dello spazio di ricerca. L'euristica fornisce una stima del costo residuo necessario per raggiungere l'obiettivo e consente di ridurre il numero di nodi esplorati, garantendo anche l'ottimalità.

In entrambi i casi, l'obiettivo è individuare il percorso che minimizza il tempo totale di percorrenza tra l'ingresso dell'ospedale e il reparto di destinazione. Gli algoritmi di ricerca vengono mantenuti invariati; ciò che varia è esclusivamente l'informazione relativa ai costi associati agli archi del grafo.

4.1 Ricerca a Costo Uniforme (Uniform Cost Search)

La ricerca a costo uniforme è un algoritmo di ricerca che seleziona i nodi da espandere in base al *costo cumulativo* del cammino dalla sorgente. In particolare, l'algoritmo espande sempre il nodo con il valore minimo di:

$$g(n)$$

dove $g(n)$ indica il costo totale del cammino dall'inizio fino al nodo n .

Dal punto di vista implementativo, la frontiera viene gestita tramite una *coda a priorità* ordinata secondo il valore di $g(n)$, in modo da garantire che il nodo con il costo minore venga sempre espanso per primo.

Test dell'obiettivo Un aspetto rilevante della ricerca a costo uniforme riguarda il momento in cui viene effettuato il test di obiettivo. Il controllo non avviene quando un nodo viene generato, ma solo quando viene estratto dalla frontiera per essere espanso. Questo meccanismo assicura che il cammino individuato verso l'obiettivo sia effettivamente quello a costo minimo.

Aggiornamento dei cammini A differenza della ricerca in ampiezza, uno stesso nodo può essere raggiunto tramite percorsi con costi differenti. Se durante l'esplorazione viene individuato un cammino più economico verso un nodo già presente nella frontiera, il costo associato a quel nodo viene aggiornato e la coda a priorità viene riordinata.

Questo comportamento rende la ricerca a costo uniforme equivalente all'algoritmo di Dijkstra, con la differenza che la ricerca termina non appena il nodo obiettivo viene estratto dalla frontiera.

Proprietà La ricerca a costo uniforme è:

- completa, se i costi delle azioni sono sempre maggiori di una quantità positiva;
- ottimale, poiché esplora i nodi in ordine di costo crescente;
- potenzialmente costosa in termini di tempo e memoria, poiché non utilizza alcuna informazione direzionale verso l'obiettivo.

Nel progetto, la ricerca a costo uniforme viene utilizzata come **baseline**, rappresentando una strategia di pianificazione basata esclusivamente sui costi delle azioni.

4.2 Algoritmo A*

L'algoritmo A* può essere visto come un'estensione della ricerca a costo uniforme, ottenuta introducendo un'informazione aggiuntiva sotto forma di una stima del costo residuo necessario per raggiungere l'obiettivo.

Per ogni nodo n , A* utilizza una funzione di valutazione definita come:

$$f(n) = g(n) + h(n)$$

dove $g(n)$ rappresenta il costo del cammino dalla sorgente al nodo n , mentre $h(n)$ indica una stima del costo minimo ancora necessario per raggiungere l'obiettivo a partire da n .

L'algoritmo seleziona per l'espansione il nodo con il valore minimo di $f(n)$, combinando il costo già sostenuto con una previsione del costo futuro.

4.3 Relazione tra A* e Ricerca a Costo Uniforme

La ricerca a costo uniforme può essere interpretata come un caso particolare dell'algoritmo A*. Quando l'euristica è nulla per tutti i nodi, ovvero:

$$h(n) = 0 \quad \forall n$$

la funzione di valutazione si riduce a:

$$f(n) = g(n)$$

In questa configurazione, A* si comporta esattamente come la ricerca a costo uniforme. Per questo motivo, nel progetto la ricerca a costo uniforme viene considerata come una versione di A* priva di euristica.

4.4 Euristica euclidea

Nel progetto, l'algoritmo A* adotta come funzione euristica la distanza euclidea tra il nodo corrente n e il nodo obiettivo g . La distanza geometrica viene calcolata come:

$$d(n, g) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$$

dove (x_n, y_n) e (x_g, y_g) rappresentano le coordinate dei due nodi all'interno della mappa dell'ospedale.

Per convertire la distanza in una stima del tempo di percorrenza, l'euristica assume una velocità di camminata *ottimistica* pari a 2.0 m/s. Questa velocità è volutamente superiore alla velocità media reale utilizzata dal simulatore, che è pari a 1.4 m/s.

Il tempo stimato dall'euristica è quindi definito come:

$$h(n) = \frac{d(n, g)}{v_{\text{eur}}} \quad \text{con} \quad v_{\text{eur}} = 2.0 \text{ m/s}$$

È importante sottolineare che l'euristica euclidea ha esclusivamente il ruolo di **guidare la ricerca**, suggerendo quali nodi risultino geometricamente più vicini all'obiettivo e, di conseguenza, più promettenti da esplorare. Essa non influisce sul valore effettivo del costo del percorso, che è determinato unicamente dalla funzione di costo associata agli archi del grafo.

In particolare, l'euristica euclidea:

- ignora completamente la struttura del grafo;
- non tiene conto della tipologia dei corridoi;
- non considera affollamento, congestione o costi dinamici;
- fornisce esclusivamente una stima geometrica del costo rimanente.

Il costo reale di un percorso è quindi determinato dalla funzione $g(n)$, che accumula i costi degli archi attraversati, mentre l'euristica $h(n)$ serve unicamente a ridurre il numero di nodi esplorati, rendendo la ricerca più efficiente.

Poiché l'euristica utilizza una velocità maggiore rispetto a quella reale e ignora tutti i fattori che possono rallentare il movimento, essa non sovrastima mai il costo reale del percorso. Di conseguenza, risulta **ammissibile** e consente ad A* di mantenere la garanzia di trovare una soluzione ottimale.

Dal punto di vista pratico, l'uso dell'euristica euclidea permette di ottenere un miglioramento significativo in termini di efficienza rispetto alla ricerca a costo uniforme, pur mantenendo inalterata la correttezza della soluzione trovata.

4.5 Ruolo degli algoritmi nel progetto

Gli algoritmi di ricerca non costituiscono l'oggetto principale di analisi del progetto, ma rappresentano il meccanismo decisionale attraverso cui l'agente pianifica il percorso. A variare è esclusivamente l'informazione sui costi fornita al processo di pianificazione.

Per quanto riguarda la strategia di ricerca, viene messo a confronto il comportamento di:

- la **ricerca a costo uniforme (UCS)**, in cui l'euristica è assente ($h(n) = 0$) e i nodi vengono espansi sulla base del solo costo cumulativo $g(n)$;
- **A* con euristica euclidea**, che utilizza una stima euristica per orientare l'esplorazione verso il nodo obiettivo.

In questo confronto, la funzione di costo $g(n)$ rimane invariata: l'euristica non modifica il valore dei percorsi, ma influisce sull'ordine con cui i nodi vengono esplorati. Questo consente di osservare come l'introduzione di un'euristica possa rendere la ricerca più efficiente, riducendo il numero di nodi espansi e il tempo di pianificazione, senza compromettere l'ottimalità della soluzione.

In modo separato, viene analizzata la qualità dell'informazione sui costi confrontando l'uso di costi statici con costi stimati tramite machine learning. In questo caso, la strategia di ricerca rimane fissa, mentre cambia la funzione $g(n)$ utilizzata dall'algoritmo.

Ciò permette di distinguere l'effetto della strategia di esplorazione da quello legato alla qualità delle stime dei costi, rendendo possibile un'analisi più precisa del comportamento complessivo dell'algoritmo di pianificazione.

5 Simulazione dei costi di percorrenza

Poiché non sono disponibili dataset pubblici che descrivano in maniera dettagliata i tempi di percorrenza all'interno di un ambiente ospedaliero indoor, i dati utilizzati nel progetto vengono generati tramite una simulazione controllata. La simulazione ha l'obiettivo di produrre tempi di percorrenza realistici, seppur semplificati, sufficienti per studiare l'impatto dell'informazione sui costi nel processo di pianificazione.

La simulazione è implementata nella classe `simulator`, che svolge il ruolo di *oracolo*: a partire dalle caratteristiche di un corridoio e dalle condizioni ambientali, restituisce il tempo reale necessario per attraversarlo. Questo valore rappresenta la *ground truth* del problema, ma non è direttamente accessibile all'algoritmo di pianificazione.

5.1 Modello del tempo di percorrenza

Il tempo di percorrenza di un corridoio viene calcolato a partire da un tempo base, determinato dalla lunghezza fisica del corridoio e da una velocità media di camminata umana. Nel simulatore, la velocità reale utilizzata è pari a **1.4 m/s**, un valore comunemente adottato come stima realistica della velocità di camminata in ambienti indoor.

Il tempo base viene successivamente modificato tramite una serie di fattori correttivi che modellano il comportamento dinamico dell'ambiente ospedaliero.

5.2 Tipo di corridoio

Ogni corridoio è associato a una specifica tipologia strutturale:

- **centrale**, tipicamente caratterizzata da un traffico più elevato;
- **secondario**, con livelli di traffico moderati;
- **isolato**, generalmente meno affollato.

La tipologia del corridoio influisce sul livello di affollamento effettivo e introduce una differenza strutturale nei tempi di percorrenza, anche a parità di condizioni ambientali.

5.3 Affollamento e congestione

Nel simulatore viene effettuata una distinzione tra *affollamento* e *congestione*, che, pur essendo concetti strettamente collegati, descrivono aspetti differenti del problema.

L'affollamento indica il numero di persone presenti in un corridoio in un determinato momento ed è modellato come un valore continuo normalizzato nell'intervallo $[0, 1]$. Valori bassi corrispondono a corridoi quasi vuoti, mentre valori vicini a 1 rappresentano situazioni con un'elevata presenza di persone.

La congestione, invece, descrive *l'effetto* che l'affollamento ha sul movimento, ovvero quanto la presenza di persone rallenta effettivamente il tempo di percorrenza. Nel simulatore, la congestione è modellata tramite un fattore di rallentamento che moltiplica il tempo base di attraversamento del corridoio.

Questo fattore di rallentamento può seguire tre modelli differenti:

- **lineare**, in cui il rallentamento cresce in modo proporzionale all'affollamento;
- **quadratico**, in cui il rallentamento aumenta in modo non lineare e diventa particolarmente marcato alle alte densità;
- **a soglia**, in cui l'affollamento non produce effetti fino al superamento di un livello critico, oltre il quale il rallentamento cresce rapidamente.

Nel progetto sono stati implementati tutti e tre, ma viene utilizzato il **modello quadratico**. Questa scelta riflette il comportamento tipico di ambienti come gli ospedali, in cui il movimento può peggiorare drasticamente quando un corridoio è già molto affollato: in tali condizioni, anche piccoli aumenti di affollamento possono causare rallentamenti significativi.

I modelli alternativi restano disponibili per l'esplorazione di scenari diversi, ma non costituiscono il focus principale della sperimentazione.

5.4 Implementazione delle componenti stocastiche

Per rendere la simulazione più realistica, il tempo di percorrenza di un corridoio non è determinato in modo completamente deterministico. Anche a parità di lunghezza, orario, affollamento e tipologia del corridoio, il tempo effettivo può variare a causa di fattori imprevedibili.

Nel simulatore, questa variabilità viene modellata introducendo due componenti stocastiche: *eventi imprevisti* e *rumore naturale*. Entrambe agiscono come fattori moltiplicativi sul tempo base.

Eventi imprevisti Gli eventi imprevisti rappresentano situazioni occasionali che rallentano temporaneamente il movimento, come la presenza di carrelli nei corridoi, gruppi di visitatori o piccole emergenze operative.

Nel simulatore, un evento si verifica con una probabilità pari al 5%. Quando l'evento si verifica, il tempo di percorrenza viene moltiplicato per un fattore di rallentamento estratto casualmente dall'intervallo [1.2, 2.0]. In assenza di eventi, il fattore è pari a 1 e non introduce alcun rallentamento.

In pratica:

- in circa 5 attraversamenti su 100 si verifica un evento imprevisto;
- in questi casi il tempo di percorrenza aumenta dal 20% fino al 100%;
- negli altri casi il tempo rimane invariato.

Rumore naturale Oltre agli eventi discreti, il simulatore introduce una forma di variabilità che considera i normali cambiamenti del comportamento umano, come variazioni nella velocità di camminata o brevi esitazioni.

Questa variabilità è modellata tramite un fattore moltiplicativo estratto da una distribuzione normale con media pari a 1.0 e deviazione standard $\sigma = 0.06$. La media pari a 1.0 garantisce che, in media, il rumore non introduca alcun bias sistematico sul tempo di percorrenza.

Con questo valore di σ :

- circa il 68% dei valori ricade nell'intervallo $\pm 6\%$;
- circa il 95% dei valori ricade nell'intervallo $\pm 12\%$;
- variazioni più estreme sono possibili, ma risultano rare.

Composizione dei fattori Il tempo totale di percorrenza di un corridoio viene ottenuto moltiplicando il tempo base per tutti i fattori deterministici e stocastici considerati: orario, affollamento, tipologia del corridoio, eventi imprevisti e rumore naturale.

Riproducibilità Poiché il simulatore utilizza componenti casuali, viene introdotto un parametro *seed* che consente di fissare i generatori di numeri casuali. Utilizzando lo stesso seed, la sequenza di eventi e di rumore viene replicata esattamente, rendendo gli esperimenti completamente riproducibili e confrontabili. Il seed rende la casualità non completamente casuale.

5.5 Costi statici

Il simulatore viene impiegato anche per costruire una versione statica dei costi associati ai corridoi, utilizzata come baseline per il confronto.

Per ciascun corridoio del grafo, il simulatore calcola 200 volte il tempo di percorrenza in diverse condizioni di orario e affollamento. A partire da questi valori viene quindi calcolata la **media** dei tempi di attraversamento.

Questa media viene utilizzata come **costo statico** del corridoio. I costi statici vengono impiegati sia nella **ricerca a costo uniforme** sia in **A* con euristica euclidea**, costituendo la funzione di costo $g(n)$ utilizzata dagli algoritmi.

In questa configurazione, i costi non dipendono dallo stato corrente dell'ambiente e rimangono invariati durante la pianificazione. I costi statici rappresentano quindi una descrizione semplificata dell'ospedale, utilizzata come mezzo di confronto per valutare l'impatto dell'uso di costi stimati tramite machine learning.

6 Generazione del dataset

Il dataset utilizzato per l'addestramento dei modelli di machine learning viene generato in modo sintetico a partire dal simulatore descritto nella sezione precedente. L'obiettivo non è riprodurre fedelmente dati reali, ma costruire un insieme di esempi controllato e bilanciato.

Ogni campione del dataset rappresenta l'attraversamento di un singolo corridoio in una specifica condizione ambientale. A ciascun campione sono associate alcune variabili osservabili, utilizzate come input del modello, e un valore target che rappresenta il tempo reale di percorrenza restituito dal simulatore.

6.1 Struttura dei campioni

Ogni esempio del dataset è composto da:

- la **lunghezza del corridoio**, che rappresenta la componente strutturale del costo;
- l'**orario**, espresso come ora del giorno, che influisce indirettamente sull'affollamento;
- il **livello di affollamento**, modellato come valore continuo normalizzato nell'intervallo $[0, 1]$;
- il **tempo reale di percorrenza**, generato dal simulatore e utilizzato come valore target.

Il modello di machine learning ha quindi il compito di apprendere una funzione che, a partire da lunghezza, orario e affollamento, fornisca una stima del tempo necessario ad attraversare un corridoio.

6.2 Campionamento stratificato

Per evitare che il dataset risulti sbilanciato verso specifiche condizioni ambientali, la generazione dei campioni avviene tramite una strategia di **campionamento stratificato**. In particolare, lo spazio delle condizioni viene suddiviso lungo due dimensioni: orario e affollamento.

La variabile temporale è suddivisa in tre fasce:

- **notte**, caratterizzata da bassa attività;
- **giorno**, con affluenza moderata;
- **fasce di picco**, tipicamente associate a maggiore affluenza.

Analogamente, l'affollamento viene suddiviso in tre livelli:

- **basso**, che rappresenta corridoi quasi vuoti;
- **medio**, che rappresenta condizioni di utilizzo normale;
- **alto**, che rappresenta situazioni di congestione.

Per ogni combinazione di fascia oraria e livello di affollamento vengono generati più campioni, in modo da coprire in maniera uniforme lo spazio delle possibili condizioni. Questo approccio riduce il rischio che il modello impari correlazioni dovute a una distribuzione dei dati non rappresentativa.

6.3 Ruolo della tipologia di corridoio

La generazione dei campioni tiene conto anche della tipologia dei corridoi presenti nel grafo. I corridoi vengono suddivisi in base alla loro categoria (strutturale), come corridoi centrali, secondari o isolati.

Durante la generazione del dataset, i corridoi vengono estratti una sola volta, evitando duplicazioni dovute alla bidirezionalità del grafo. In questo modo, ogni corridoio contribuisce al dataset in maniera equa e controllata, mantenendo una corretta rappresentazione delle diverse tipologie strutturali.

6.4 Mescolamento e indipendenza dei campioni

Una volta generati, i campioni vengono mescolati casualmente. Questa operazione serve a rompere l'ordinamento artificiale introdotto dal processo di generazione, mantenendo però invariata la distribuzione dei dati.

Senza il mescolamento, campioni con caratteristiche molto simili (ad esempio stessa fascia oraria e stesso livello di affollamento) risulterebbero consecutivi nel dataset. In questa situazione, il modello potrebbe apparire particolarmente accurato semplicemente perché osserva esempi molto simili uno dopo l'altro, senza aver realmente appreso una relazione generale tra le variabili.

Il mescolamento riduce quindi il rischio di **overfitting strutturale**, ovvero il caso in cui il modello sembra performare bene solo perché sfrutta regolarità dovute all'ordine dei dati, e non perché ha imparato una funzione di stima robusta.

L'operazione di shuffle preserva comunque l'associazione tra input e target, garantendo che ogni campione rimanga corretto.

6.5 Suddivisione in training e test

Il dataset finale viene suddiviso in un **training set** e in un **test set** attraverso una suddivisione casuale controllata. Il training set è utilizzato per l'addestramento dei modelli di machine learning, mentre il test set viene impiegato esclusivamente per valutare le prestazioni su dati non visti durante l'addestramento.

In particolare, il 20% dei campioni viene assegnato al test set. La suddivisione è controllata tramite un parametro di inizializzazione (*seed*), che garantisce la riproducibilità degli esperimenti e consente di ottenere la stessa partizione dei dati in esecuzioni diverse.

Questo approccio consente di mantenere il processo di addestramento semplice.

6.6 Obiettivo del dataset

Il dataset generato non ha lo scopo di fornire previsioni perfette, ma di consentire ai modelli di machine learning di catturare le principali relazioni tra struttura del corridoio, condizioni ambientali e tempo di percorrenza.

Questo dataset costituisce quindi il ponte tra la simulazione del mondo reale e la pianificazione dei percorsi: i modelli addestrati su questi dati vengono poi utilizzati per stimare dinamicamente i costi degli archi durante la ricerca del percorso.

7 Modelli di machine learning per la stima dei costi

Nel progetto il machine learning viene utilizzato per stimare il tempo di percorrenza dei corridoi dell'ospedale in condizioni dinamiche. L'idea non è quella di costruire un modello predittivo perfetto, ma di ottenere stime ragionevoli dei costi, sufficientemente realistiche da poter essere usate dall'algoritmo di pianificazione A^* .

In altre parole, il modello di machine learning non prende decisioni sul percorso finale, ma fornisce ad A^* una funzione di costo alternativa rispetto ai costi statici. Questo permette di studiare quando l'uso di costi appresi dai dati può portare vantaggi e quando invece può condurre ad errori.

7.1 Input e output dei modelli

Tutti i modelli considerati stimano il tempo necessario ad attraversare un singolo corridoio a partire da tre variabili:

- la **lunghezza** del corridoio;
- l'**orario** della giornata;
- il livello di **affollamento**.

L'output del modello è un singolo valore reale che rappresenta il tempo di percorrenza stimato. Questa stima viene poi utilizzata come costo dell'arco durante la ricerca del percorso.

7.2 Regressione lineare

Il primo modello utilizzato è una regressione lineare. In questo caso si assume che il tempo di percorrenza possa essere approssimato come una combinazione lineare delle variabili di input. Formalmente, la stima del tempo $\hat{t}$ è data da:

$$\hat{t} = w_0 + w_1 \cdot L + w_2 \cdot H + w_3 \cdot A$$

dove L è la lunghezza del corridoio, H rappresenta l'orario e A il livello di affollamento.

Ogni coefficiente indica quanto una variabile contribuisce alla stima finale. Ad esempio, un coefficiente elevato associato all'affollamento indica che il modello ritiene questo fattore particolarmente rilevante nel determinare il tempo di percorrenza.

La regressione lineare è semplice, veloce da addestrare e facile da interpretare. Tuttavia, il suo principale limite è l'incapacità di rappresentare comportamenti non lineari, come quelli che emergono in presenza di forte congestione.

7.3 Random Forest

Per superare questi limiti viene utilizzato anche un modello basato su Random Forest. Questo approccio utilizza un insieme di alberi decisionali, ognuno dei quali impara una diversa suddivisione dello spazio dei dati.

Ogni albero fornisce una stima del tempo di percorrenza, e il risultato finale è ottenuto come media delle stime dei singoli alberi. In questo modo il modello è in grado di catturare relazioni non lineari e interazioni complesse tra lunghezza, orario e affollamento.

Un aspetto interessante della Random Forest è la possibilità di analizzare la **feature importance**, cioè una misura di quanto ciascuna variabile di input contribuisca alle decisioni del modello. In altre parole, permette di capire su quali fattori il modello si basa di più per stimare il tempo di percorrenza di un corridoio.

Questa analisi fornisce indicazioni utili anche sul comportamento del sistema simulato, evidenziando quali caratteristiche (ad esempio lunghezza del corridoio, orario o livello di affollamento) risultano più rilevanti nel determinare i tempi di attraversamento.

7.4 Valutazione delle prestazioni

Dopo l'addestramento, i modelli vengono valutati su un insieme di test separato, contenente dati che non sono stati utilizzati durante la fase di apprendimento. Questo permette di misurare quanto bene il modello generalizza a situazioni nuove.

Le prestazioni vengono misurate utilizzando diverse metriche:

- **MAE**, che indica di quanto, in media, il modello sbaglia in secondi;
- **RMSE**, che penalizza maggiormente gli errori più grandi;
- **R²**, che misura quanta parte della variabilità dei dati viene spiegata dal modello;
- **MAPE**, che esprime l'errore medio in percentuale.

7.5 Integrazione con l'algoritmo di pianificazione

Una volta addestrati, i modelli vengono integrati nel processo di pianificazione attraverso la costruzione di funzioni di costo dinamiche. Date le condizioni correnti dell'ambiente, come orario e affollamento, il modello fornisce una stima del costo di ciascun corridoio.

Questa stima viene utilizzata da A^* al posto dei costi statici, permettendo di studiare come l'uso di informazioni apprese dai dati influenzi la qualità del percorso trovato e l'efficienza della ricerca.

In questo contesto, il machine learning agisce intermediario tra la simulazione del mondo reale e l'algoritmo di pianificazione: A^* non conosce i costi reali, ma prende decisioni basate su stime, rendendo possibile un'analisi più realistica del problema.

8 Metriche di valutazione

Per valutare il comportamento dell'algoritmo di pianificazione nelle diverse configurazioni di costo (statico, statico con euristica, ML lineare, ML Random Forest) non è sufficiente osservare il solo percorso finale. È necessario analizzare sia la qualità della soluzione trovata, sia l'efficienza computazionale della ricerca.

Per questo motivo vengono definite diverse metriche, calcolate a livello di singolo test e successivamente aggregate su più esecuzioni.

8.1 Metriche a livello di singolo percorso

Ogni esecuzione di A^* produce un percorso che viene analizzato in dettaglio. Le metriche principali considerate sono le seguenti.

Costo stimato e costo reale Il **costo stimato** è il valore utilizzato da A^* durante la pianificazione. Questo costo dipende dalla configurazione adottata: nel nostro caso può derivare da costi statici medi oppure da stime fornite da un modello di machine learning.

Il **costo reale** rappresenta invece il tempo effettivo di percorrenza del percorso, calcolato utilizzando il simulatore come oracolo. Questo valore costituisce la *ground truth* ed è noto solo a posteriori, dopo che il percorso è stato trovato.

Errore di stima Per misurare quanto la pianificazione si basi su stime accurate, viene calcolato l'errore tra costo stimato e costo reale:

$$\text{Errore assoluto} = |c_{\text{stimato}} - c_{\text{reale}}|$$

e l'errore relativo, ottenuto normalizzando l'errore rispetto al costo reale.

Questa metrica è particolarmente importante per le configurazioni basate su machine learning, poiché quantifica direttamente la qualità delle predizioni del modello nel contesto della pianificazione.

Ottimalità della soluzione Per ogni scenario viene calcolato anche il **percorso ottimo reale**, ottenuto utilizzando A^* con accesso diretto ai costi reali del simulatore. Questo percorso rappresenta il miglior risultato teoricamente possibile.

Il confronto tra il percorso trovato e quello ottimo viene espresso tramite il **gap di ottimalità**:

$$\text{Gap} = \frac{c_{\text{reale}} - c_{\text{ottimo}}}{c_{\text{ottimo}}}$$

Un gap pari a zero indica che l'algoritmo ha trovato una soluzione ottima, mentre valori positivi indicano una degradazione della qualità del percorso.

Efficienza computazionale Oltre alla qualità del percorso, vengono misurate anche le prestazioni computazionali della ricerca:

- il numero di **nodi espansi**, che indica quante volte A^* ha effettivamente esplorato uno stato;
- il **tempo di esecuzione**, misurato in secondi.

Queste metriche permettono di valutare l'impatto di euristiche e stime dei costi sulla complessità della ricerca.

8.2 Metriche aggregate

Poiché i singoli test dipendono da condizioni casuali (orario, affollamento, eventi imprevisti), ogni configurazione viene valutata su un insieme di test indipendenti.

A partire dalle metriche dei singoli percorsi vengono calcolate statistiche aggregate, tra cui:

- media e deviazione standard del costo reale;
- media, deviazione standard e valore massimo del gap di ottimalità;
- percentuale di soluzioni ottime (gap inferiore all'1%);
- media e deviazione standard dei nodi espansi;
- media e deviazione standard del tempo di esecuzione;
- media e deviazione standard dell'errore di stima.

Queste statistiche permettono di confrontare le configurazioni non solo in termini di prestazioni medie, ma anche di stabilità e robustezza.

8.3 Confronto tra configurazioni

Le metriche aggregate vengono utilizzate per confrontare:

- l'effetto dell euristica euclidea rispetto alla ricerca a costo uniforme;
- l'impatto dell'uso di costi stimati tramite machine learning rispetto ai costi statici;
- le differenze tra modelli di machine learning semplici (regressione lineare) e modelli più complessi (Random Forest).

In particolare, l'analisi si concentra su due aspetti fondamentali:

- **qualità della soluzione**, valutata tramite costo reale e gap di ottimalità;
- **efficienza della ricerca**, valutata tramite nodi espansi e tempo di esecuzione.

8.4 Quando il machine learning è vantaggioso

Infine, viene condotta un'analisi comparativa diretta tra configurazioni statiche e configurazioni basate su machine learning, mantenendo identiche le condizioni di test. Per ogni scenario si verifica se il machine learning:

- trova un percorso con costo reale inferiore;
- espande un numero minore di nodi;
- introduce penalità significative rispetto all'utilizzo del costo statico.

Questo consente di individuare le condizioni in cui l'uso di stime apprese dai dati risulta effettivamente utile, e quelle in cui invece la semplicità dei costi statici rappresenta una scelta più robusta.

8.5 Scenari sperimentali e ambiente di test

I test sono stati condotti su un grafo di 26 nodi e su due scenari distinti, pensati per simulare livelli diversi di dinamicità all'interno di un ambiente ospedaliero.

Lo **scenario normale** rappresenta una situazione abbastanza stabile, con una variabilità moderata dei tempi di percorrenza. In questo caso, la probabilità di eventi imprevisti (come rallentamenti temporanei o congestioni) è fissata al 3%, simulando condizioni di una giornata ordinaria.

Lo **scenario estremo** introduce invece un livello di incertezza molto più elevato. La probabilità di eventi casuali sale al 10%, rendendo i tempi di percorrenza più instabili e meno prevedibili. Questo scenario è

pensato per riprodurre situazioni di forte affollamento o di stress operativo, in cui i costi statici risultano meno affidabili.

Per ciascuno scenario sono stati eseguiti 50 test di pianificazione, confrontando quattro configurazioni diverse: A^* con costi statici, A^* con costi statici ed euristica euclidea, A^* con costi stimati tramite regressione lineare e A^* con costi stimati tramite Random Forest.

8.6 Scenario normale

Nello scenario a variabilità moderata, l'approccio statico si dimostra già una soluzione solida. Il costo reale medio dei percorsi trovati è di circa 79.5s, con un gap medio rispetto al percorso ottimo reale pari a circa 6.8% e una percentuale di soluzioni ottime del 44%. Il numero di nodi espansi rimane costante (25 in media), indicando un comportamento stabile e prevedibile dell'algoritmo.

L'introduzione dell'euristica euclidea riduce in modo netto il numero di nodi esplorati (circa 18 in media), ma questo miglioramento in efficienza non si traduce in una migliore qualità della soluzione. Al contrario, il costo medio aumenta fino a circa 85.8s e la percentuale di soluzioni ottime scende al 22%. In uno scenario poco dinamico, quindi, una guida basata solo sulla distanza geometrica può risultare ingannevole rispetto al tempo reale di percorrenza.

La configurazione basata su regressione lineare non riesce a migliorare il baseline statico. Il costo medio si attesta intorno a 84.6s, con un gap medio di circa 9.2% e una percentuale di soluzioni ottime pari al 22%. Questo risultato è coerente con i limiti del modello lineare, che fatica a catturare le interazioni non lineari tra lunghezza dei corridoi, orario e livello di affollamento.

La Random Forest è la configurazione che ottiene i risultati migliori anche in questo scenario. Il costo medio scende a circa 78.1s, con un gap medio del 2.4% e una percentuale di soluzioni ottime pari al 56%. Inoltre, il numero medio di nodi espansi si riduce a circa 23, indicando una ricerca più mirata rispetto al baseline statico. Il miglioramento in qualità avviene però a fronte di un tempo computazionale molto più elevato, dovuto alla valutazione del modello di machine learning durante la pianificazione.

8.7 Scenario estremo

Nello scenario estremo, caratterizzato da un'elevata variabilità dei costi, le differenze tra le varie configurazioni risultano ancora più evidenti. L'approccio statico ottiene un costo reale medio di circa 83.5s, con un gap medio del 5.2% e una percentuale di soluzioni ottime pari al 40%. Anche in questo caso il numero di nodi espansi rimane costante, mostrando una limitata capacità di adattamento alle condizioni dinamiche.

L'uso dell'euristica euclidea consente una riduzione significativa del numero di nodi esplorati (circa 19 in media), ma peggiora sensibilmente la qualità delle soluzioni. Il costo medio sale infatti a circa 90.2s, mentre il gap e l'errore di stima aumentano. Questo risultato evidenzia come, in ambienti molto dinamici, una ricerca più veloce dal punto di vista computazionale non garantisca necessariamente un percorso migliore in termini di tempo reale.

La regressione lineare mostra prestazioni intermedie. Il costo medio è di circa 86.6s, con un gap medio del 4.1% e una percentuale di soluzioni ottime del 42%. Sebbene migliori leggermente il gap rispetto allo statico, il modello non riesce comunque a sfruttare appieno l'informazione dinamica disponibile.

La Random Forest conferma invece il proprio vantaggio in condizioni di alta incertezza. Il costo medio scende a circa 82.2s, con un gap medio di appena 0.8%, il più basso tra tutte le configurazioni. La percentuale di soluzioni ottime sale al 52% e il numero medio di nodi espansi si riduce a circa 22, indicando una ricerca più adattiva. Anche in questo scenario, il miglioramento della qualità delle soluzioni avviene a fronte di un aumento significativo del tempo computazionale.

Oltre ai risultati di pianificazione riportati in Tabella 1 e 2, sono stati analizzati anche i dati relativi alle prestazioni dei modelli di machine learning utilizzati per stimare il tempo di percorrenza dei corridoi.

Per valutare la qualità delle stime vengono utilizzate diverse metriche. Il **MAE** (Mean Absolute Error) rappresenta l'errore medio assoluto tra il tempo stimato dal modello e il tempo reale: indica quindi, in media, di quanti secondi il modello sbaglia la previsione. Il **RMSE** (Root Mean Squared Error) misura anch'esso l'errore di stima, ma penalizza maggiormente gli errori più grandi. Il coefficiente R^2 indica invece quanto il modello riesce a spiegare la variabilità dei dati: valori vicini a 1 indicano una

Table 1: Riepilogo dei risultati medi sui due scenari

Configurazione	Costo medio (s)	Gap ottimo (%)	Soluzioni ottime (%)	Nodi espansi	Tempo (ms)
Scenario normale					
Statico	79.47	6.8	44.0	25.0	0.07
Statico + Euclidea	85.81	10.4	22.0	18.0	0.07
ML Lineare	84.57	9.2	22.0	25.5	4.58
ML Random Forest	78.13	2.4	56.0	22.6	1327.12
Scenario estremo					
Statico	83.47	5.2	40.0	25.0	0.07
Statico + Euclidea	90.19	7.7	40.0	19.0	0.07
ML Lineare	86.63	4.1	42.0	25.6	4.63
ML Random Forest	82.18	0.8	52.0	21.6	1269.73

Table 2: Statistiche aggiuntive sui risultati

Configurazione	Std costo (s)	Costo min (s)	Costo max (s)	Errore stima medio (s)	Tempo medio (ms)
Scenario normale					
Statico	28.20	51.54	165.77	21.91	0.07
Statico + Euclidea	38.44	38.68	205.69	31.12	0.07
ML Lineare	37.13	38.50	188.13	15.43	4.58
ML Random Forest	32.77	38.13	194.09	13.67	1327.12
Scenario estremo					
Statico	28.96	51.74	186.94	22.35	0.07
Statico + Euclidea	41.86	36.84	213.45	32.77	0.07
ML Lineare	39.04	40.94	194.20	18.40	4.63
ML Random Forest	31.34	38.14	165.22	19.90	1269.73

buona capacità predittiva, mentre valori vicini a 0 indicano prestazioni limitate. Infine, il **MAPE** misura l'errore medio in termini percentuali rispetto al valore reale.

Nel caso della regressione lineare, le prestazioni risultano piuttosto limitate. Nello scenario normale il modello presenta un MAE sul test set di circa 6.8s, il che significa che, in media, la stima del tempo di percorrenza sbaglia di quasi sette secondi. Il valore di R^2 , pari a circa 0.27, indica che il modello riesce a spiegare solo una parte ridotta della variabilità dei dati. Nello scenario estremo le prestazioni peggiorano ulteriormente, con un MAE che supera gli 8s e un valore di R^2 intorno a 0.23, segno di una scarsa capacità di adattarsi a situazioni molto dinamiche.

La Random Forest mostra risultati migliori. Nello scenario normale il MAE sul test set scende a circa 5.4s, indicando stime mediamente più accurate rispetto alla regressione lineare. Anche il valore di R^2 è più elevato (circa 0.38), a conferma di una maggiore capacità del modello di catturare le dinamiche del problema. Nello scenario estremo il MAE aumenta, arrivando a circa 7.5s, ma il modello continua comunque a fornire stime più affidabili rispetto al modello lineare.

Un aspetto interessante riguarda l'analisi della **feature importance** della Random Forest. I risultati mostrano che il livello di affollamento è la variabile più rilevante, con un peso di circa il 60% in entrambi gli scenari. Questo significa che il modello basa la maggior parte delle sue decisioni su quanto il corridoio è affollato. La lunghezza del corridoio e l'orario hanno un peso minore e simile tra loro, intorno al 20% ciascuno.

Considerando i singoli test di pianificazione, la Random Forest ottiene un costo reale inferiore rispetto all'approccio statico nel 62% dei casi nello scenario normale e nel 60% dei casi nello scenario estremo. Inoltre, il modello riesce a trovare il percorso ottimo nel 56% dei test nello scenario normale e nel 52% di quelli estremi. Negli altri casi non si osservano miglioramenti significativi, o si introduce una penalità, indicando che il machine learning non è sempre la scelta migliore, ma risulta più efficace quando il contesto è complesso e caratterizzato da forte variabilità.

La Tabella 2 riassume alcune statistiche aggiuntive che aiutano a interpretare meglio i risultati principali. In particolare, la deviazione standard e i valori minimo e massimo del costo mostrano come, in presenza di alta variabilità, i tempi di percorrenza possano cambiare molto da un test all'altro, soprattutto negli scenari estremi.

L'errore medio di stima evidenzia inoltre che i modelli di machine learning, in particolare la Random Forest, riescono a stimare i costi dei corridoi in modo più accurato rispetto agli approcci statici. Questo contribuisce a spiegare la riduzione del gap rispetto all'ottimo osservata negli scenari più complessi.

Infine, i tempi medi di pianificazione confermano il principale compromesso dell'approccio basato su machine learning: a fronte di stime più accurate e percorsi mediamente migliori, il costo computazionale cresce in modo significativo rispetto alle configurazioni statiche.

8.8 Discussione complessiva e risposta alla domanda iniziale

L'obiettivo di questo lavoro era capire *quando* ha davvero senso usare il machine learning all'interno di A^* e quando, invece, un approccio statico è già più che sufficiente. I risultati mostrano abbastanza chiaramente che il machine learning non è sempre la scelta migliore: funziona bene solo in alcune situazioni specifiche.

Negli scenari con **variabilità moderata**, l'approccio statico si difende molto bene. I costi medi dei percorsi sono vicini all'ottimo reale, il gap resta contenuto e i tempi di calcolo sono quasi nulli. In questo contesto, aggiungere il machine learning non porta grandi vantaggi. La regressione lineare, in particolare, tende spesso a peggiorare le soluzioni, mentre la Random Forest migliora solo leggermente il costo medio e il gap, ma al prezzo di un tempo di pianificazione molto più alto. In pratica, quando l'ambiente è abbastanza stabile, il modello statico è più semplice, più veloce e già sufficientemente efficace.

Le cose cambiano quando si passa a scenari con **alta variabilità**. Qui i tempi di percorrenza dipendono molto di più dall'affollamento e da eventi imprevedibili, e i costi statici, basati su valori medi, non riescono più a descrivere bene la situazione reale. In questo caso la Random Forest mostra un vantaggio più chiaro: il gap rispetto all'ottimo reale si riduce in modo significativo e la percentuale di soluzioni ottime aumenta, superando il 50% dei casi. Questo indica che il machine learning diventa davvero utile quando l'ambiente è abbastanza complesso da rendere poco affidabili le stime statiche.

Va però detto che questo miglioramento ha un costo. Usare la Random Forest significa aumentare in modo significativo il tempo di pianificazione. Il miglioramento nella qualità del percorso va quindi sempre messo a confronto con il costo computazionale, soprattutto in applicazioni in cui è importante rispondere rapidamente.

L'euristica euclidea, invece, si conferma utile per rendere la ricerca più efficiente, riducendo il numero di nodi esplorati. Tuttavia, i risultati mostrano anche che, in ambienti molto dinamici, esplorare meno nodi non garantisce automaticamente un percorso migliore. In altre parole, una buona euristica dal punto di vista geometrico non è sempre una buona euristica dal punto di vista temporale.

In sintesi, i risultati permettono di rispondere in modo diretto alla domanda iniziale:

- l'approccio **statico** è la scelta più sensata in ambienti stabili o poco variabili, dove garantisce buone soluzioni con tempi di calcolo minimi;
- il **machine learning**, in particolare la Random Forest, diventa utile in scenari più complessi e variabili, dove la capacità di adattarsi alle condizioni correnti permette di avvicinarsi di più all'ottimo reale;
- l'uso del ML ha senso solo quando il bisogno di adattività giustifica l'aumento del costo computazionale.

Nel complesso, questi risultati suggeriscono che il machine learning non debba sostituire gli approcci classici, ma affiancarli quando serve. Gli approcci tradizionali restano la scelta migliore quando l'ambiente è prevedibile, mentre il ML diventa uno strumento davvero utile solo quando la complessità e l'incertezza rendono necessario adattare dinamicamente i costi.

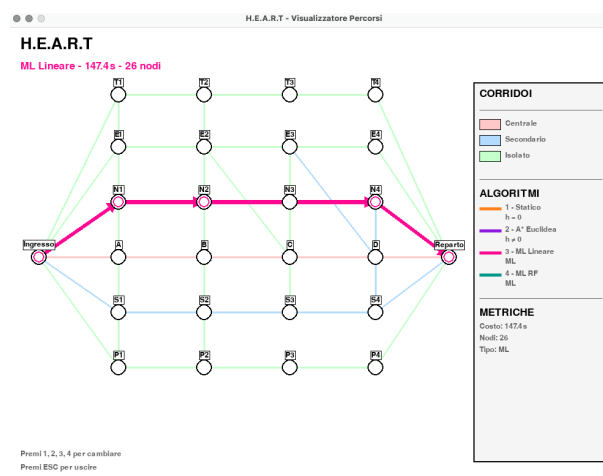
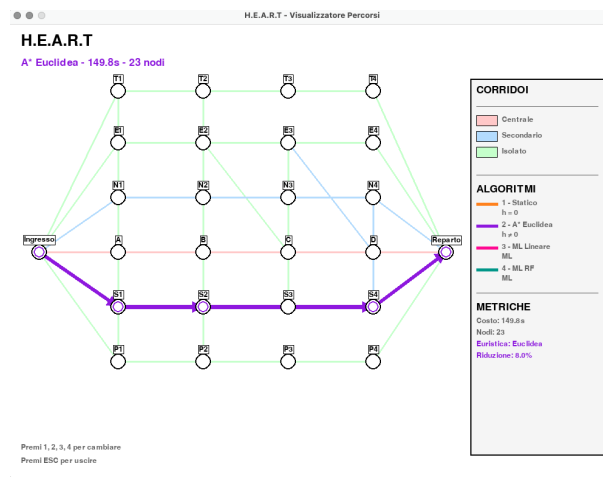
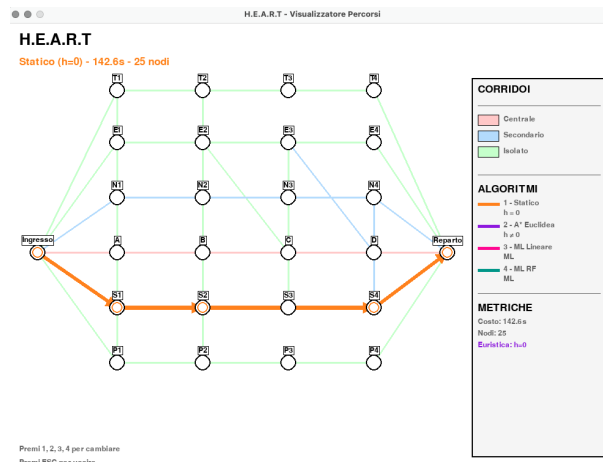
8.9 Visualizzazione e demo interattiva

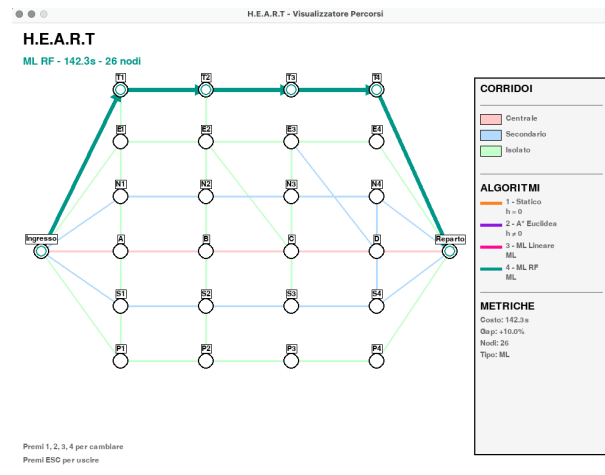
Per rendere più chiari i risultati ottenuti è stata realizzata una demo interattiva che mostra il grafo dell'ospedale, il percorso scelto dall'algoritmo di pianificazione e alcune metriche di riferimento per le diverse configurazioni di costo.

La demo permette di confrontare direttamente l'uso di costi statici, A^* con euristica euclidea e stime basate su machine learning. In molti casi i percorsi coincidono: questo indica che il modello ha imparato

correttamente la struttura del grafo e non introduce deviazioni inutili.

8.9.1 Esempio Demo: Ore 14:00, Affollamento:90%





Le differenze tra i percorsi, in modo coerente ai risultati, si notano soprattutto quando vengono simulate situazioni di forte congestione. In questi casi l'informazione dinamica porta l'algoritmo a scegliere percorsi alternativi, mostrando un comportamento più adattivo rispetto all'uso di costi statici.

Il grafo utilizzato è volutamente piccolo, così da rendere immediato il confronto tra le diverse strategie. La demo non sostituisce l'analisi quantitativa, ma aiuta a capire meglio come le diverse stime di costo influenzano la pianificazione. La demo simula casualmente nuove condizioni ad ogni avvio.